```
  def westernLine =
          Line.named("Western").departingFrom("Emu Plains")

  timetableService.findLinesThrough("Parramatta",
                                    "Town Hall") >> [westernLine]

  timetableService.findArrivalTimes(westernLine, "Parramatta") >>
      [at(7,58), at(8,00), at(8,02), at(8,11), at(8,14), at(8,21)]

  when:
  def proposedTrainTimes =
                  itineraryService.findNextDepartures("Parramatta",
                                                      "Town Hall",
                                                      at(8,00));

  then:
  proposedTrainTimes == [at(8,02), at(8,11), at(8,14)]
}
```

❶ **The pretend timetable service returns this train line when you ask which lines go through Parramatta and Town Hall.**

Define a train line.

❷ **The pretend timetable service also gives you the correct arrival times at Parramatta.**

This sets up a "pretend" (or "mock") timetable service to model what you need the real timetable service to do. You know that it needs to tell you what lines go through any two stations ❶ and what time trains on a given line get to a particular station ❷. The `>>` sign in Spock is shorthand for saying "when I call this method with these parameters, return these values."

For this to compile, you need a `TimetableService`. In Java, you'd typically define an interface. This lets you put off actually implementing the `TimetableService` until later, after you've finished with the `ItineraryService` class. You defined the methods the `TimetableService` needs in ❶ and ❷, so the interface might look like this:

```
package com.bddinaction.chapter2.services;

import com.bddinaction.chapter2.model.Line;
import org.joda.time.LocalTime;

import java.util.List;

public interface TimetableService {
    List<LocalTime> findArrivalTimes(Line line, String targetStation);
    List<Line> findLinesThrough(String departure, String destination);
}
```

The final piece in the puzzle is the `findArrivalTimesAfter()` method, which returns a list of departure times after a given time, as shown in this possible implementation:

```
private List<LocalTime> findArrivalTimesAfter(LocalTime startTime,
                                              List<LocalTime> times) {
    List<LocalTime> viableArrivalTimes = Lists.newArrayList();
    for(LocalTime arrivalTime : times) {
        if (arrivalTime.isAfter(startTime)) {
            viableArrivalTimes.add(arrivalTime);
        }
    }
    return viableArrivalTimes;
}
```